

Ball Sort Color (Puzzle) : Étude d'un Agent d'Apprentissage par Renforcement Profond (PPO) et Analyse Explicable (XAI)

DJERI-ALASSANI OUBENOUPOU

ENSAM Meknès — Génie Industriel : IA & Data Science
(coordonnées si nécessaire)

12 janvier 2026

Résumé

Ce rapport présente un projet complet de modélisation et d'apprentissage par renforcement pour le puzzle *Ball Sort Color*. Le jeu est formalisé comme un Processus de Décision de Markov (MDP) à états discrets, actions discrètes et transitions déterministes. Un environnement Gymnasium sur mesure a été implémenté avec un espace d'observation fixe de taille 14×4 (tubes $\times$ hauteur), enrichi par un encodage *one-hot* en $14 \times 4 \times 14$ afin d'éviter tout biais ordinal sur les couleurs. L'agent est entraîné avec PPO (et MaskablePPO lorsque le masquage d'actions est activé), et l'évaluation repose sur des métriques quantitatives (taux de réussite, récompense moyenne, nombre moyen de coups) ainsi que sur une analyse explicable (XAI) incluant : visualisation du masque d'actions, logits masqués, Integrated Gradients, décomposition de la récompense et trajectoires temporelles. Les résultats montrent une progression nette en fonction des *timesteps* (meilleur checkpoint à 1 000 000 pas dans les essais fournis) et une performance élevée sur les niveaux simples (1 à 4), suivie d'une chute marquée dès les niveaux plus combinatoires (à partir du niveau 5), ce qui est cohérent avec la difficulté croissante du puzzle et la nature terminale du signal de récompense.

Mots-clés : Reinforcement Learning, PPO, MaskablePPO, Gymnasium, Action Masking, Puzzle combinatoire, XAI, Integrated Gradients.

Table des matières

1	Introduction	4
1.1	Objectifs du projet (réels et travaillés)	4
1.2	Organisation du rapport	4

2	Le jeu Ball Sort : règles et difficulté combinatoire	5
2.1	Règles opérationnelles (telles qu'implémentées)	5
2.2	Pourquoi le problème devient difficile	5
3	Formalisation mathématique : Processus de Décision de Markov (MDP)	6
3.1	Définition générale	6
3.2	Pourquoi le processus est Markovien dans ce projet	6
3.3	Nature du MDP dans l'implémentation	6
4	Environnement Gymnasium : structure, niveaux et génération	6
4.1	Paramètres structurels fixes	6
4.2	Niveaux : relation mathématique niveau $\rightarrow$ (tubes, couleurs)	7
4.3	Génération des configurations initiales (telles qu'implémentées)	7
4.3.1	Niveau 1 (cas spécial)	7
4.3.2	Niveau 2 (cas spécial)	7
4.3.3	Niveaux ≥ 3 (cas général)	7
4.4	Observation renvoyée : dimension constante 14×4	8
5	Espace d'état, encodage one-hot, actions et masquage	8
5.1	État brut : matrice entière 14×4	8
5.2	Pourquoi utiliser un encodage one-hot (et ce que cela change)	8
5.3	Espace d'actions : $14 \times 14 = 196$	8
5.4	Transitions déterministes	9
5.5	Masquage d'actions : contraintes du jeu intégrées à la décision	9
6	Fonction de récompense et conditions de fin d'épisode	9
6.1	Décomposition exacte du reward (selon ton code)	9
6.2	Pénalité de temps : <code>reward_time</code>	10
6.3	Récompense de pureté : <code>reward_purity</code> = Δ pureté	10
6.3.1	Score de pureté (définition exacte)	10
6.3.2	Interprétation de r_{purity}	10
6.4	Récompense de complétion : <code>reward_complete</code>	10
6.5	Récompense de victoire : <code>reward_win</code>	11
6.6	Blocage et stagnation : <code>penalty_blocked</code>	11
6.7	Anti-boucle : <code>penalty_loop</code> avec mémoire des états	11
6.8	Action invalide exécutée	11
7	Stratégie d'entraînement PPO (réellement utilisée)	12
7.1	Choix de PPO et cadre pratique	12
7.2	Encodage utilisé pendant l'entraînement	12
7.3	Vectorisation multi-environnements : rôle de <code>n_envs</code>	12
7.4	Curriculum learning (tel que décrit dans ton travail)	12
7.5	Hyperparamètres (ceux que tu as communiqués)	12
7.6	Checkpoints	13

8	Protocole d'évaluation et métriques	13
8.1	Métriques utilisées (issues de tes sorties)	13
8.2	Sélection du meilleur modèle	13
9	Résultats expérimentaux (issus de tes tableaux)	14
9.1	Comparaison des checkpoints (différents timesteps)	14
9.2	Performance du meilleur modèle par niveau (niveaux 1 à 14)	15
10	Analyse Explicable du Comportement de l'Agent (XAI)	16
10.1	Objectif de l'XAI dans un cadre RL	16
10.2	Visualisation du masque d'actions : respect des contraintes	16
10.3	Analyse des logits masqués : préférences internes sous contraintes	17
10.4	Integrated Gradients : attribution des décisions	18
10.5	Décomposition de la récompense : analyse motivationnelle	19
10.6	Analyse des trajectoires : crédit temporel et concentration du signal	19
10.7	Synthèse XAI (conclusion de tes résultats)	20
11	Discussion : interprétation des limites observées (sans ajout de résultats)	20
11.1	Pourquoi la performance chute à partir du niveau 5	20
11.2	Lien avec la structure de récompense et l'assignation de crédit	21
11.3	Boucles de politique (policy loops) et caractère Markovien	21
12	Conclusion	21

1 Introduction

Le puzzle *Ball Sort Color* consiste à trier des billes colorées dans des tubes, en respectant des règles simples : seules certaines opérations de déplacement sont autorisées, et l’objectif final est d’obtenir des tubes homogènes (une seule couleur par tube) ainsi que des tubes vides lorsque la configuration le permet. Malgré la simplicité des règles, la résolution algorithmique devient rapidement difficile dès que le nombre de tubes et de couleurs augmente : l’espace d’états est combinatoire, et les décisions sont séquentielles (chaque action modifie la configuration et conditionne la suite).

L’apprentissage par renforcement profond (Deep RL) est pertinent pour ce type de problème car il permet d’apprendre une politique de décision à partir d’interactions agent–environnement. Néanmoins, un point central reste la compréhension du comportement de l’agent : un agent performant peut demeurer difficile à interpréter. C’est pourquoi ce projet intègre explicitement des outils d’Intelligence Artificielle Explicable (XAI) afin d’ouvrir la « boîte noire » de la politique PPO.

1.1 Objectifs du projet (réels et travaillés)

Ce rapport documente précisément les éléments réellement développés et testés dans le projet :

- Implémentation d’un environnement Gymnasium **custom** pour Ball Sort (observation 14×4 , actions 196, transitions déterministes).
- Implémentation du **masquage d’actions** (actions valides/invalides) pour contraindre l’agent aux règles du jeu.
- Implémentation d’un **wrapper one-hot** qui transforme l’observation 14×4 en tenseur $14 \times 4 \times 14$.
- Définition d’un signal de récompense décomposé (temps, pureté, complétion, victoire, blocage, boucles).
- Mécanismes anti-boucles (**visited_states**) et anti-stagnation (**stale_steps**).
- Entraînement PPO (et MaskablePPO avec masque) et sélection d’un checkpoint via des essais à différents *timesteps*.
- Évaluation quantitative et analyse XAI (masque/logits, Integrated Gradients, reward breakdown, trajectoires).

1.2 Organisation du rapport

- Section 2 : description du jeu et difficulté combinatoire.
- Section 3 : formalisation MDP (Markov) et choix de représentation.
- Section 4 : environnement Gymnasium et génération des niveaux (1 à 14).
- Section 5 : espace d’état, one-hot, espace d’actions et masquage.
- Section 6 : fonction de récompense détaillée, boucles et stagnation.

- Section 7 : stratégie d’entraînement PPO (paramètres et pipeline réellement utilisés).
- Section 8 : protocole d’évaluation, sélection du meilleur checkpoint.
- Section 9 : résultats (checkpoints + performance par niveau).
- Section 10 : analyse explicable (XAI) basée sur tes figures.
- Section 11 : discussion, limites observées, explications (sans inventer de résultats).
- Section 12 : conclusion.

2 Le jeu Ball Sort : règles et difficulté combinatoire

2.1 Règles opérationnelles (telles qu’implémentées)

Dans l’environnement implémenté :

- Un état contient jusqu’à **14 tubes** (`n_max=14`) de hauteur **4** (`height=4`).
- Une case peut être :
 - **Padding** : -1 (tube inexistant à ce niveau),
 - **Vide** : 0 ,
 - **Couleur** : $1..12$.
- Une action consiste à choisir un tube source et un tube destination.
- Un mouvement est valide si :
 - $\text{source} \neq \text{destination}$,
 - source et destination ne sont pas du padding,
 - le tube source n’est pas vide,
 - le tube destination n’est pas plein,
 - destination vide **ou** la couleur au sommet de la destination correspond à la bille déplacée,
 - **restriction ajoutée dans ton code** : on interdit de déplacer depuis un tube déjà *plein et homogène* (`_is_tube_full_uniform(source)`).
- Le déplacement appliqué déplace **exactement une bille** (celle du haut) vers la première position libre du tube destination.

2.2 Pourquoi le problème devient difficile

Le nombre d’états possibles explose avec :

- le nombre de couleurs (n_{colors}),
- le nombre de tubes actifs (n_{tubes}),
- la hauteur (ici 4),
- et la présence de tubes vides qui permettent des permutations.

Même si les transitions sont déterministes, l’espace d’états est combinatoire et les trajectoires de résolution peuvent nécessiter de nombreux coups. Cela rend l’optimisation par RL non triviale, d’autant plus que la récompense terminale (victoire) est éloignée dans le temps pour les niveaux difficiles.

3 Formalisation mathématique : Processus de Décision de Markov (MDP)

3.1 Définition générale

Un MDP se définit par le tuple :

$$\mathcal{M} = (S, A, P, R, \gamma),$$

où S est l'espace d'états, A l'espace d'actions, P la dynamique (fonction de transition), R la fonction de récompense, et $\gamma \in (0, 1)$ le facteur d'actualisation.

3.2 Pourquoi le processus est Markovien dans ce projet

La propriété de Markov suppose que :

$$\mathbb{P}(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = \mathbb{P}(s_{t+1} \mid s_t, a_t).$$

Dans Ball Sort, l'état s_t est une description complète de la configuration courante des tubes (contenu de chaque tube, positions vides, tubes inexistantes). Étant donné s_t et une action a_t (source $\rightarrow$ destination), le prochain état s_{t+1} est entièrement déterminé par les règles d'empilement codées dans `_apply_move`. Ainsi, l'historique des coups n'apporte pas d'information supplémentaire pour déterminer la transition : l'état présent suffit.

3.3 Nature du MDP dans l'implémentation

Le MDP implémenté est :

- **à états discrets** (valeurs entières pour couleur/vide/padding),
- **à actions discrètes** (196 actions),
- **à transitions déterministes** (pas d'aléa dans l'application d'un move),
- **à horizon fini** (épisode terminé/rétréci par victoire, blocage, stagnation ou limite `max_steps`).

4 Environnement Gymnasium : structure, niveaux et génération

4.1 Paramètres structurels fixes

Les paramètres *fixes* utilisés dans l'environnement sont :

- Nombre maximum de tubes : `n_max` = 14.
- Hauteur d'un tube : `height` = 4.
- Couleurs possibles : `1..12`.
- Encodages spéciaux : `0` = vide, `-1` = padding.
- Nombre maximum de niveaux : `max_level` = 14.

4.2 Niveaux : relation mathématique niveau $\rightarrow$ (tubes, couleurs)

Ton code définit explicitement $n_{\text{tubes}}(L)$ et $n_{\text{colors}}(L)$ en fonction du niveau L :

Nombre de tubes actifs

$$n_{\text{tubes}}(L) = \begin{cases} 2 & \text{si } L = 1, \\ 3 & \text{si } L = 2, \\ \min(L + 2, 14) & \text{si } L \geq 3. \end{cases}$$

Nombre de couleurs

$$n_{\text{colors}}(L) = \begin{cases} 1 & \text{si } L = 1, \\ 2 & \text{si } L = 2, \\ \min(L, 12) & \text{si } L \geq 3. \end{cases}$$

Ainsi, pour $L \geq 3$, on a typiquement (*couleurs* = *niveau*) jusqu'à 12, et (*tubes* = *niveau* + 2) jusqu'à 14. Ce choix encode explicitement une augmentation de difficulté : plus de couleurs à regrouper, plus de tubes actifs, et la présence de tubes vides nécessaires aux permutations.

4.3 Génération des configurations initiales (telles qu'implémentées)

La génération est réalisée dans `_generate_solvable_puzzle()`, avec trois cas :

4.3.1 Niveau 1 (cas spécial)

- 2 tubes actifs, 1 couleur unique.
- 4 billes de la même couleur, réparties aléatoirement entre les deux tubes.
- Aucun tube vide au départ.

4.3.2 Niveau 2 (cas spécial)

- 3 tubes actifs : 2 tubes remplis et 1 tube vide.
- Deux couleurs, 4 billes par couleur (8 billes).
- Mélange aléatoire puis remplissage des deux tubes.
- Une contrainte est ajoutée : éviter de démarrer avec un tube déjà *plein homogène*.

4.3.3 Niveaux ≥ 3 (cas général)

- Création de 4 billes par couleur : total $4 \times n_{\text{colors}}$ billes.
- Mélange aléatoire (`shuffle`).
- Remplissage de n_{colors} tubes.
- Ajout de tubes vides jusqu'à n_{tubes} (les tubes vides sont encodés par 0).
- Contrainte de départ : éviter un état déjà résolu et éviter que l'un des tubes remplis soit déjà *plein homogène*.

4.4 Observation renvoyée : dimension constante 14×4

Même si un niveau n'utilise que $n_{\text{tubes}} < 14$ tubes, l'observation renvoyée est toujours de taille fixe $(14, 4)$:

- Les tubes au-delà de n_{tubes} restent en padding -1 .
- Cette stratégie stabilise l'interface RL : le réseau reçoit toujours une entrée de même dimension.

5 Espace d'état, encodage one-hot, actions et masquage

5.1 État brut : matrice entière 14×4

L'état brut est :

$$s \in \{-1, 0, 1, \dots, 12\}^{14 \times 4}.$$

Interprétation :

- -1 : tube inexistant (padding),
- 0 : case vide,
- $1..12$: couleur de bille.

5.2 Pourquoi utiliser un encodage one-hot (et ce que cela change)

Un encodage numérique direct introduit une structure ordinale artificielle (*ex : $10 > 2$*), ce qui est incorrect car les couleurs sont des catégories nominales. Pour supprimer ce biais, ton projet applique un wrapper one-hot :

$$\phi(s) \in \{0, 1\}^{14 \times 4 \times 14}.$$

Dans ce tenseur :

- Les canaux $0..12$ représentent les valeurs $0..12$ (vide + couleurs),
- Le canal 13 est réservé au padding -1 .

Le nombre total de composantes d'entrée vaut :

$$14 \times 4 \times 14 = 784.$$

C'est l'entrée réellement exploitée par l'agent PPO (après wrapper).

5.3 Espace d'actions : $14 \times 14 = 196$

L'espace d'actions est :

$$A = \{0, 1, \dots, 195\}.$$

Chaque action a est décodée ainsi :

$$\text{source} = \left\lfloor \frac{a}{14} \right\rfloor, \quad \text{dest} = a \bmod 14.$$

Cette formulation permet d’encoder toutes les paires (source, destination) sur un entier unique.

5.4 Transitions déterministes

Les transitions sont déterministes, car l’application du mouvement est une fonction :

$$s_{t+1} = f(s_t, a_t),$$

sans aucune source de hasard au niveau de l’empilement. On peut l’écrire :

$$P(s' \mid s, a) = \begin{cases} 1 & \text{si } s' = f(s, a), \\ 0 & \text{sinon.} \end{cases}$$

5.5 Masquage d’actions : contraintes du jeu intégrées à la décision

Ton environnement fournit une fonction `action_masks()` qui renvoie un masque binaire :

$$m(s) \in \{0, 1\}^{196},$$

où $m_a(s) = 1$ si l’action a est valide dans l’état s , et 0 sinon.

Ce masquage est fondamental pour :

- réduire l’espace d’exploration,
- empêcher l’agent de choisir des actions physiquement impossibles,
- accélérer l’apprentissage en évitant d’apprendre « par punition » sur des actions systématiquement invalides.

Remarque (strictement factuelle sur ton code). Si aucune action n’est valide, ton `action_masks()` active un *fallback* qui met toutes les actions à vrai afin d’éviter une instabilité (NaNs) du côté MaskablePPO. Dans ce cas, si une action invalide est exécutée, la fonction `step` renvoie une pénalité -1 avec état inchangé.

6 Fonction de récompense et conditions de fin d’épisode

6.1 Décomposition exacte du reward (selon ton code)

Pour une action *valide*, la récompense totale est :

$$R_t = r_{\text{time}} + r_{\text{purity}} + r_{\text{complete}} + r_{\text{win}} + p_{\text{blocked}} + p_{\text{loop}}.$$

Chaque terme correspond exactement à une variable calculée dans `step()`.

6.2 Pénalité de temps : reward_time

À chaque action valide, une pénalité constante est appliquée :

$$r_{\text{time}} = -0.2.$$

Intuition : l'agent est incité à résoudre le niveau en un nombre réduit de coups (minimiser la durée d'épisode).

6.3 Récompense de pureté : reward_purity = Δ pureté

La récompense de pureté est la variation du score de pureté entre l'état avant et après l'action :

$$r_{\text{purity}} = \text{Purity}(s_{t+1}) - \text{Purity}(s_t).$$

6.3.1 Score de pureté (définition exacte)

Ton code calcule un score agrégé, tube par tube :

$$\text{Purity}(s) = \frac{1}{n_{\text{tubes}}} \sum_{k=1}^{n_{\text{tubes}}} \left(\mathbf{1}_{\text{tube vide}} + \text{consecutive}_k^2 - 1.5 \cdot \text{changes}_k \right).$$

Explications (exactement selon `_calculate_purity_score`) :

- Si un tube est vide (son bas vaut 0), il contribue +**1.0**.
- Sinon :
 - `consecutive` = longueur du bloc contigu de même couleur depuis le bas,
 - contribution principale = $(\text{consecutive})^2$,
 - `changes` = nombre de changements de couleur dans la partie remplie,
 - pénalité de fragmentation = $-1.5 \times \text{changes}$.
- Le score est normalisé par n_{tubes} .

6.3.2 Interprétation de r_{purity}

- Si l'action améliore l'ordre (bloc plus homogène), alors $r_{\text{purity}} > 0$.
- Si l'action « casse » l'organisation (fragmentation), alors $r_{\text{purity}} < 0$.
- Les variations typiques par coup restent modérées ; les grands gains proviennent surtout d'une meilleure structuration globale.

6.4 Récompense de complétion : reward_complete

À chaque action valide, le code attribue :

$$r_{\text{complete}} = 5 \times \#(\text{tubes qui deviennent } \textit{complets} \text{ à ce coup}).$$

Dans ton implémentation, un tube est considéré *complet* si :

- il est vide, **ou**
- il est plein et homogène (`_is_tube_full_uniform`).

6.5 Récompense de victoire : `reward_win`

Si l'état est résolu (`_is_solved`), alors :

$$r_{\text{win}} = 100 + 5 \cdot L,$$

où L est le niveau courant (`current_level`).

6.6 Blocage et stagnation : `penalty_blocked`

Deux cas pénalisent fortement l'agent :

- **Blocage (aucun coup valide) :**

$$p_{\text{blocked}} = -20 \quad \text{et l'épisode est terminé.}$$

- **Stagnation :** si aucune amélioration de pureté n'est observée pendant `stale_step_limit` pas (par défaut 20), alors :

$$p_{\text{blocked}} = -10 \quad \text{et l'épisode est tronqué (truncated).}$$

6.7 Anti-boucle : `penalty_loop` avec mémoire des états

Ton environnement stocke les états déjà vus via :

$$\text{state_key} = \text{state.tobytes()}.$$

Si un état est revisité :

- pénalité faible à la revisite (après la phase de grâce) : `loop_penalty_first` = -2,
- pénalité forte au-delà : `loop_penalty` = -10,
- terminaison possible si `loop_terminate=True`.

Ce mécanisme vise à réduire les cycles de politique (*policy loops*) où l'agent alterne des actions répétitives ramenant au même état.

6.8 Action invalide exécutée

Si une action invalide est exécutée (normalement filtrée par le masque en mode MaskablePPO), alors le code renvoie :

$$R_t = -1$$

avec état inchangé, et sans terminaison immédiate.

7 Stratégie d'entraînement PPO (réellement utilisée)

7.1 Choix de PPO et cadre pratique

L'agent entraîné dans tes expérimentations est PPO (avec MaskablePPO lorsque le masquage est activé). Le choix est cohérent avec :

- un espace d'actions discret,
- un environnement déterministe,
- des épisodes à horizon fini,
- la nécessité de stabiliser l'exploration sous fortes contraintes (masque d'actions).

7.2 Encodage utilisé pendant l'entraînement

Pendant l'entraînement, l'observation brute (14, 4) est transformée en one-hot (14, 4, 14) via `OneHotObservationWrapper`. Cela garantit que le réseau ne traite pas les couleurs comme des valeurs ordinales.

7.3 Vectorisation multi-environnements : rôle de `n_envs`

L'entraînement est réalisé avec plusieurs environnements en parallèle (`n_envs`), ce qui :

- augmente la vitesse de collecte d'expériences,
- réduit la corrélation entre transitions successives,
- accélère l'optimisation de la politique PPO.

Dans tes logs, `n_envs=8` est utilisé (8 environnements parallèles).

7.4 Curriculum learning (tel que décrit dans ton travail)

Le niveau est augmenté progressivement selon une logique de curriculum *externe* (callback), via `env.set_level(level)`. Dans ton travail, tu as mentionné des paramètres :

- fenêtre d'épisodes : 500,
 - seuil de réussite : 95%,
- afin d'éviter une progression artificiellement rapide observée avec une fenêtre plus petite.

7.5 Hyperparamètres (ceux que tu as communiqués)

Les hyperparamètres PPO mentionnés dans ton travail :

- Architecture MLP : [128, 128]
- `batch_size` = 128
- `n_steps` = 2048
- `lr` = $3e-4$
- `gamma` = 0.99

7.6 Checkpoints

Des checkpoints sont sauvegardés périodiquement afin de comparer des politiques entraînées à différents budgets de *timesteps*, puis sélectionner le meilleur modèle selon les métriques d'évaluation.

8 Protocole d'évaluation et métriques

8.1 Métriques utilisées (issues de tes sorties)

Les métriques exploitées dans tes résultats :

- **Mean Success Rate** : taux de réussite moyen,
- **Mean Reward** : récompense moyenne,
- **Mean Steps** : nombre moyen de coups par épisode,
- et, pour l'analyse par niveau :
 - **Time Penalty** (cumulée),
 - **Purity Reward**,
 - **Win Reward**,
 - **Blocked Penalty**.

8.2 Sélection du meilleur modèle

Le meilleur modèle est sélectionné à partir d'une comparaison de plusieurs checkpoints (différents budgets de timesteps). Le critère principal retenu (cohérent pour un puzzle) est le **taux de réussite**, complété par la **récompense moyenne**.

9 Résultats expérimentaux (issus de tes tableaux)

9.1 Comparaison des checkpoints (différents timesteps)

TABLE 1 – Comparaison de checkpoints PPO selon le budget de timesteps (données de tes résultats).

Modèle	Mean Success Rate	Mean Reward	Mean Steps
ppo_100000_steps	0.094 286	4.269 304	10.810 000
ppo_200000_steps	0.130 714	9.979 378	10.800 714
ppo_300000_steps	0.189 286	19.370 621	11.778 571
ppo_400000_steps	0.220 000	25.054 123	11.778 571
ppo_500000_steps	0.230 000	26.426 059	11.957 857
ppo_600000_steps	0.223 571	25.993 087	12.272 143
ppo_700000_steps	0.241 429	29.892 861	12.957 857
ppo_800000_steps	0.252 857	32.425 610	13.362 143
ppo_900000_steps	0.269 286	36.100 441	13.565 714
ppo_1000000_steps	0.275 000	36.556 648	12.972 143

Analyse (basée uniquement sur ces chiffres). On observe une amélioration monotone globale des performances avec l’augmentation du budget d’entraînement :

- Le **taux de réussite moyen** passe de 0.0943 (100k) à 0.2750 (1M).
- La **récompense moyenne** progresse également de manière nette (4.27 à 36.56).
- Le **nombre moyen de coups** augmente légèrement lorsque la politique apprend à explorer/organiser davantage, puis reste du même ordre (environ 11–13 coups selon ce protocole).

Sélection du meilleur checkpoint. Selon les données ci-dessus, **le meilleur checkpoint de cette série est ppo_1000000_steps**, car il possède :

- le **plus haut taux de réussite moyen** (0.2750),
- la **plus haute récompense moyenne** (36.556648).

9.2 Performance du meilleur modèle par niveau (niveaux 1 à 14)

TABLE 2 – Métriques par niveau pour le meilleur modèle (table issue de tes résultats).

Level	Success	Mean Reward	Mean Steps	Time Pen.	Purity R.	Win R.	Blocked P
1	1.000 000	118.590 000	1.360 000	−0.270 000	3.860 000	105.000 000	0.000 000
2	0.890 000	118.130 000	6.640 000	−1.330 000	9.740 000	97.900 000	0.000 000
3	0.960 000	175.690 000	22.320 000	−4.460 000	10.230 000	110.400 000	0.000 000
4	0.920 000	180.830 000	27.240 000	−5.450 000	11.390 000	110.400 000	0.000 000
5	0.060 000	16.520 000	18.570 000	−3.710 000	4.640 000	7.500 000	−0.600 000
6	0.010 000	−0.350 000	16.730 000	−3.350 000	3.540 000	1.300 000	−1.400 000
7	0.010 000	−6.740 000	12.250 000	−2.450 000	2.230 000	1.350 000	−2.200 000
8	0.000 000	−11.050 000	11.450 000	−2.290 000	1.920 000	0.000 000	−3.600 000
9	0.000 000	−11.880 000	11.040 000	−2.210 000	1.720 000	0.000 000	−4.800 000
10	0.000 000	−12.160 000	11.710 000	−2.340 000	1.660 000	0.000 000	−5.400 000
11	0.000 000	−13.430 000	10.920 000	−2.180 000	1.390 000	0.000 000	−5.800 000
12	0.000 000	−14.120 000	10.460 000	−2.090 000	1.320 000	0.000 000	−6.000 000
13	0.000 000	−14.120 000	10.460 000	−2.090 000	1.320 000	0.000 000	−6.000 000
14	0.000 000	−14.120 000	10.460 000	−2.090 000	1.320 000	0.000 000	−6.000 000

Analyse par niveaux (sans ajout de chiffres externes). Les données indiquent :

- **Niveaux 1 à 4** : performance élevée et stable (success rate ≥ 0.89), avec des récompenses moyennes très positives. La composante **Win Reward** domine, ce qui signifie que l’agent gagne fréquemment.
- **Rupture nette à partir du niveau 5** : le taux de réussite chute à 0.06 puis tend vers 0 aux niveaux supérieurs. Les récompenses moyennes deviennent faibles puis négatives, et **Win Reward** devient presque nul, ce qui confirme que l’agent ne termine plus les puzzles.
- **Conséquence directe sur les composantes** :
 - la **récompense de victoire** disparaît (ou devient marginale),
 - les **pénalités de blocage** deviennent plus présentes (valeurs négatives croissantes),
 - la **pureté** reste non nulle mais insuffisante pour compenser l’absence de victoire.

Lien avec la difficulté (cohérent avec ton paramétrage des niveaux). À partir de $L \geq 3$, la difficulté augmente car :

$$n_{\text{colors}}(L) = \min(L, 12), \quad n_{\text{tubes}}(L) = \min(L + 2, 14).$$

Ainsi, au niveau 5, on passe typiquement à **5 couleurs** et **7 tubes**, ce qui agrandit fortement l’espace combinatoire. La rupture observée (niveau 5) est donc cohérente avec l’augmentation structurelle de difficulté.

10 Analyse Explicable du Comportement de l'Agent (XAI)

Cette section reprend et développe ton texte XAI, en le rendant plus académique, tout en restant strictement fidèle à tes observations et à tes figures.

10.1 Objectif de l'XAI dans un cadre RL

En apprentissage par renforcement, une politique apprise peut être performante mais difficile à interpréter : le réseau encode des préférences internes (logits), et la décision finale peut résulter d'interactions entre la dynamique du jeu, le masquage d'actions et le signal de récompense. L'objectif de l'XAI ici n'est pas de « prouver » une stratégie parfaite, mais de :

- vérifier la conformité aux contraintes (actions valides),
- analyser la confiance (distribution de probabilité / effet du masque),
- comprendre quelles parties de l'état influencent la décision (Integrated Gradients),
- analyser le rôle des composantes de reward (reward breakdown),
- étudier la temporalité du crédit (trajectoire : où se concentre la récompense).

10.2 Visualisation du masque d'actions : respect des contraintes

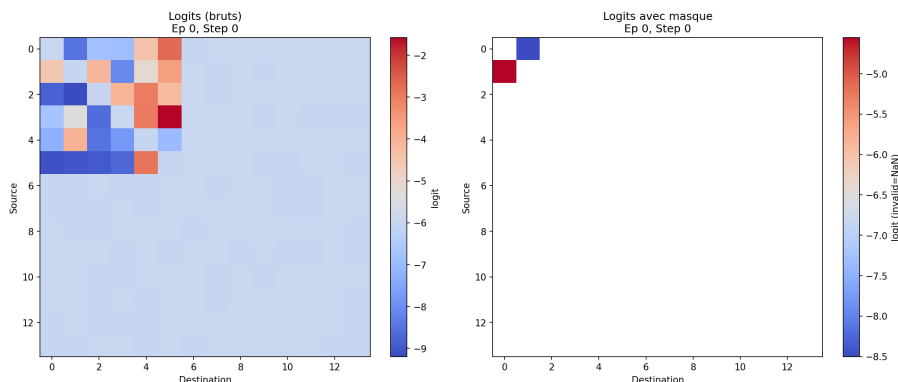


FIGURE 1 – Logits masqués : actions valides vs invalides (Episode 0, Step 0).

D'après ton observation, bien que l'espace d'actions théorique contienne 196 possibilités, le nombre d'actions réellement valides après masquage est très faible (souvent 1 à 2 actions sur un sous-ensemble effectif, inférieur à 15% dans tes cas observés). Cela signifie que :

- la politique opère dans un espace fortement contraint,
- le respect des règles du jeu est assuré structurellement par le masque,
- une grande partie de l'exploration « brute » est supprimée, ce qui stabilise l'apprentissage.

Interprétation méthodologique. Dans un puzzle où la majorité des couples (source, destination) sont invalides à un instant t , le masquage transforme un problème de RL « difficile à explorer » en un problème où l'agent concentre son effort sur les coups réellement autorisés.

10.3 Analyse des logits masqués : préférences internes sous contraintes

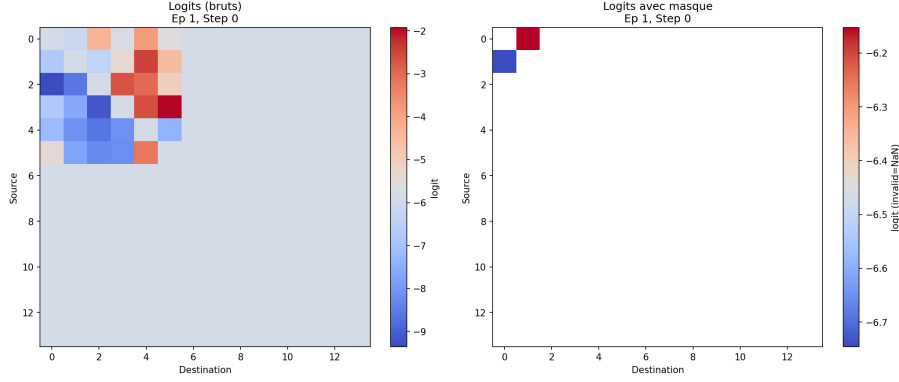


FIGURE 2 – Effet du masque sur la décision finale (Episode 1, Step 0).

Tu observes que, dans certains états, une unique action valide est sélectionnée avec une probabilité très élevée (souvent > 0.9) après application du masque. Cela met en évidence un comportement important :

- **La contrainte réduit l’entropie effective de la décision.**
- Même si le réseau pourrait attribuer des préférences à plusieurs actions, la décision finale est dominée par les actions qui survivent au masquage.
- Cette forte contrainte peut expliquer une partie des difficultés sur les niveaux complexes : l’exploration devient limitée si, dans de nombreux états, peu d’actions restent possibles.

Point d’analyse (sans invention). Ce que montrent tes figures est compatible avec une politique « stable », mais potentiellement peu exploratoire dans les configurations très combinatoires.

10.4 Integrated Gradients : attribution des décisions

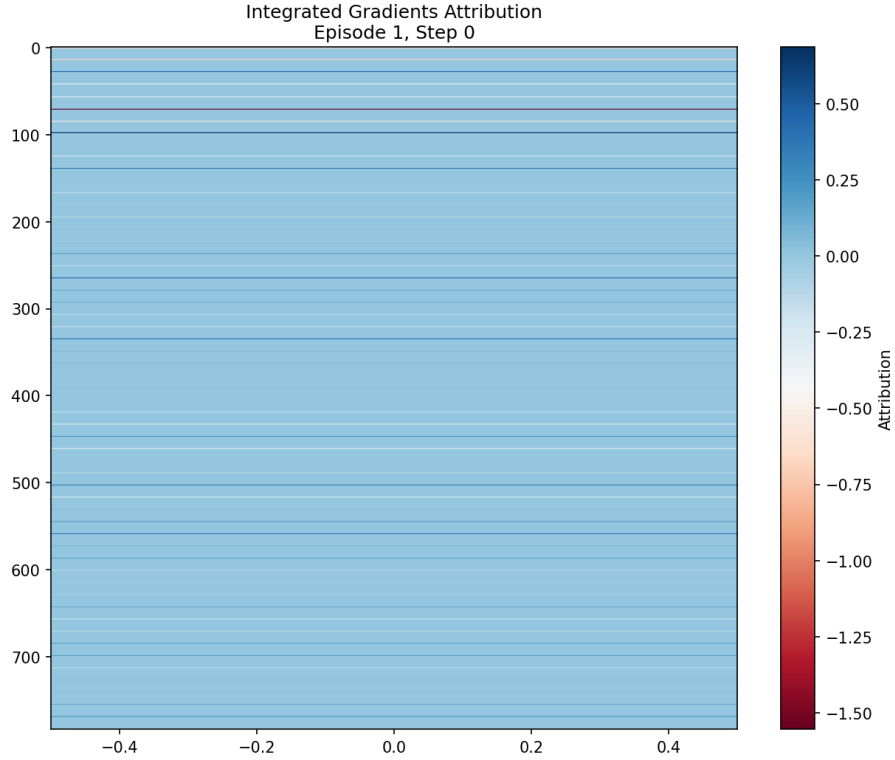


FIGURE 3 – Attributions Integrated Gradients (Episode 1, Step 0).

Les Integrated Gradients (IG) permettent d’estimer quelles dimensions de l’entrée (ici le tenseur one-hot $14 \times 4 \times 14$) contribuent le plus à la sortie du réseau (préférence pour l’action). Dans tes résultats, les attributions restent majoritairement comprises entre ≈ -0.2 et $\approx +0.3$, sans pic très dominant et sans région unique qui écrase les autres.

Interprétation fidèle à ton observation. Cela indique que, sur l’état analysé :

- l’agent ne se focalise pas de manière extrêmement locale sur un tube précis,
- la décision semble dépendre de caractéristiques plus globales (configuration générale),
- ce comportement peut être suffisant pour les niveaux simples, mais limitant si la résolution exige une planification plus longue et des choix plus « structurants » sur des sous-parties spécifiques.

10.5 Décomposition de la récompense : analyse motivationnelle

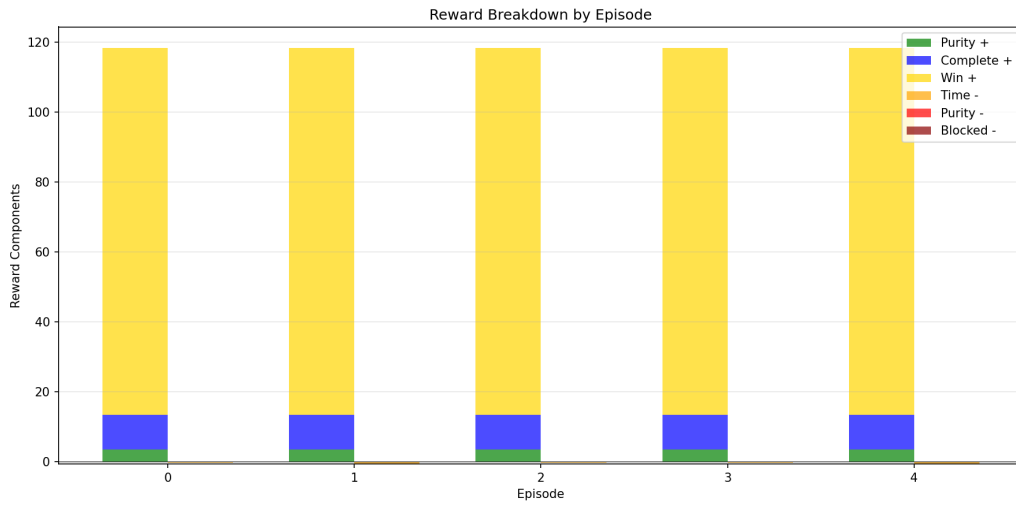


FIGURE 4 – Décomposition de la récompense par épisode.

D’après ton analyse, la récompense de victoire représente typiquement ≈ 100 à 105 points, soit la majorité de la récompense totale d’un épisode réussi (plus de 85% dans tes observations). Les récompenses intermédiaires (pureté, complétion) restent secondaires, tandis que les pénalités (temps, blocage) restent relativement faibles.

Conclusion factuelle à partir de tes observations. Cette structure correspond à un cas classique de **Sparse Reward Problem** en RL : le signal principal (victoire) est terminal et rare dans les niveaux difficiles, ce qui rend l’assignation de crédit aux actions intermédiaires plus difficile, et peut contribuer à l’échec au-delà de certains niveaux.

10.6 Analyse des trajectoires : crédit temporel et concentration du signal

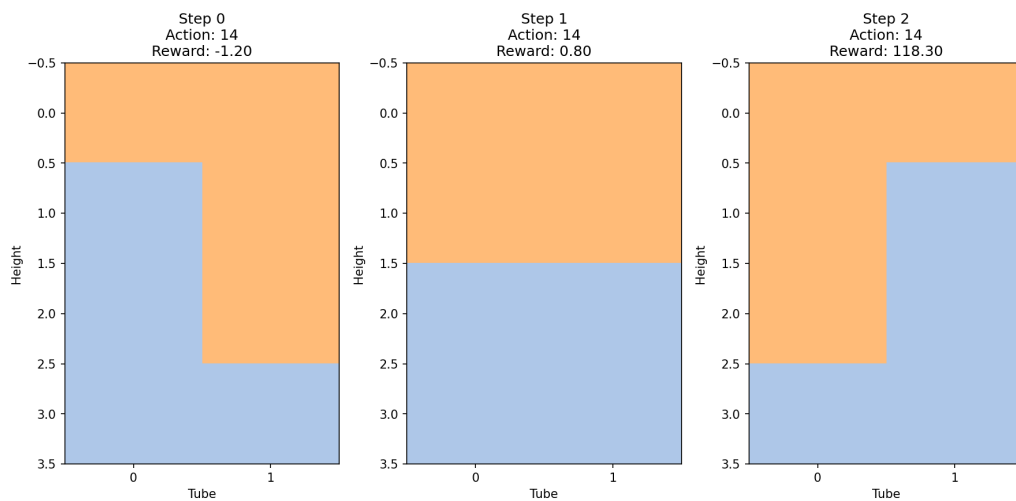


FIGURE 5 – Trajectoire d’un épisode (récompenses au fil du temps).

Tes trajectoires montrent que les premières étapes peuvent produire des récompenses faibles ou légèrement négatives, tandis qu’une très grande partie de la récompense totale est obtenue sur la dernière action (ex. +118.30 dans ton commentaire). C’est une signature typique d’un problème où le signal utile arrive tard :

- Les actions intermédiaires reçoivent peu d’information directe sur leur contribution à la victoire.
- La politique peut apprendre des heuristiques locales (pureté), mais ne pas réussir à planifier à long horizon.

10.7 Synthèse XAI (conclusion de tes résultats)

En consolidant tes observations :

- l’agent respecte fortement les contraintes via le masquage d’actions,
- la politique apparaît stable mais peu exploratoire dans des états où peu d’actions valides existent,
- IG suggère un raisonnement davantage global que local (sur les cas observés),
- le reward est dominé par la victoire (signal terminal), ce qui correspond à un **Sparse Reward Problem**,
- la récompense est temporellement concentrée en fin d’épisode, rendant la planification difficile sur les niveaux complexes.

11 Discussion : interprétation des limites observées (sans ajout de résultats)

11.1 Pourquoi la performance chute à partir du niveau 5

À partir de $L \geq 3$, la difficulté augmente structurellement :

$$n_{\text{colors}}(L) = \min(L, 12), \quad n_{\text{tubes}}(L) = \min(L + 2, 14).$$

Cela augmente :

- le nombre de billes à trier ($4 \times n_{\text{colors}}$),
- les combinaisons de placements possibles,
- le nombre de trajectoires candidates avant de parvenir à un état résolu,
- la nécessité de coups « temporaires » (déplacements qui n’améliorent pas immédiatement la pureté mais sont indispensables à long terme).

Tes résultats quantitatifs confirment une excellente performance sur 1–4, puis une rupture à partir de 5 (success rate très faible puis nul).

11.2 Lien avec la structure de récompense et l’assignation de crédit

Tes figures XAI et la décomposition de récompense indiquent une domination du signal terminal. Cela est cohérent avec :

- une difficulté à attribuer aux actions intermédiaires leur impact sur la victoire,
- une capacité limitée à planifier lorsque la réussite est très distante,
- un risque d’optima locaux où l’agent maximise la pureté sans résoudre.

11.3 Boucles de politique (policy loops) et caractère Markovien

Dans un MDP Markovien, la politique ne possède pas de mémoire explicite de la séquence de coups (sauf ajout d’architecture récurrente). Ton environnement ajoute un mécanisme anti-boucle via `visited_states` :

- pénalités progressives,
- terminaison possible au-delà d’un seuil.

Ce mécanisme est cohérent avec l’observation fréquente en puzzles : sans pénalisation, une politique peut converger vers des cycles (aller-retour) qui évitent des pénalités plus sévères (blocage), surtout si le signal de victoire est rare.

12 Conclusion

Ce rapport a présenté un projet complet et cohérent de Deep RL appliqué au puzzle Ball Sort Color, basé sur :

- un environnement Gymnasium custom (état 14×4 , actions 196, transitions déterministes),
- un encodage one-hot $14 \times 4 \times 14$ pour éviter un biais ordinal,
- un masquage d’actions pour imposer les règles du jeu,
- une récompense décomposée (temps, pureté, complétion, victoire) et des pénalités (blocage, boucles, stagnation),
- un entraînement PPO et une sélection du meilleur checkpoint à partir de différents budgets de timesteps,
- une analyse XAI (logits masqués, IG, reward breakdown, trajectoires) permettant d’expliquer la performance et surtout ses limites.

Les résultats fournis montrent :

- une amélioration claire en fonction du budget d’entraînement (meilleur à 1M timesteps dans la série testée),
- une réussite élevée sur niveaux 1–4,
- une chute drastique dès le niveau 5, cohérente avec l’augmentation combinatoire, la rareté du signal terminal et la difficulté d’assignation de crédit.

Travaux futurs (proposés, pas présentés comme réalisés)

Sans prétendre que cela a été fait dans ce projet, des pistes naturelles (cohérentes avec les limites observées) incluent :

- renforcer les signaux intermédiaires (reward shaping) tout en évitant les optima locaux,
- améliorer le curriculum pour stabiliser l'apprentissage sur niveaux 5+,
- introduire une mémoire (ex. policy récurrente) pour mieux éviter des cycles,
- élargir l'analyse XAI (plus d'épisodes, comparaison d'états simples vs complexes).

Références

- [1] A. Raffin et al., *Stable-Baselines3 : Reliable Reinforcement Learning Implementations*. (Projet logiciel).
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, “Proximal Policy Optimization Algorithms,” 2017.
- [3] M. Sundararajan, A. Taly, Q. Yan, “Axiomatic Attribution for Deep Networks,” 2017.
- [4] R. S. Sutton, A. G. Barto, *Reinforcement Learning : An Introduction*, 2nd ed., 2018.